



Proposal - Four Seasons General Merchandise

Project ID - APPLUS10112025

Date - 11/10/2025

Four Seasons General Merchandise



Applify Inc

600 Stewart Street,
Suite 400, Seattle
WA 98101

About Four Seasons General Merchandise

Four Seasons General Merchandise is a large, long-running wholesale distributor serving thousands of B2B buyers across categories such as household goods, cosmetics, seasonal items, toys, and general merchandise. With a wide SKU range, active customer base across retail and wholesale channels, and a distribution model dependent on high-volume ordering, order accuracy, and timely fulfilment, operational efficiency and customer support are critical.

The company currently runs a mix of legacy systems—including Sage ERP, SAP dashboards, SQL Server databases, and in-house applications—which limits their ability to automate workflows, enable real-time visibility, or deliver consistent digital experiences.

4SGM's leadership is now focused on modernizing a part of the ecosystem, starting with a strong AI foundation using AWS to improve customer engagement, reduce manual processes, and prepare for future phases such as automated onboarding, document verification, and personalized shopping assistance.

Executive Summary

Four Seasons General Merchandise wants to implement an AI-powered chatbot and a foundational knowledge system using Amazon Bedrock. The initial objective is simple:

Deliver a reliable, controlled, compliant chatbot that answers basic customer questions about policies, and general inquiries.

Future phases will expand the chatbot into a richer assistant capable of:

- pulling real-time order information
- offering product suggestions
- optimizing freight decisions
- generating offer bundles
- assisting in customer onboarding
- automating verification workflows

While the core product is envisioned as a unified chatbot embedded into the homepage of the 4SGM ecommerce site, the underlying architecture must be designed to scale to these future capabilities.

Applify will build a strong AI foundation, leveraging Bedrock, OpenSearch, and AWS-native services, while keeping in mind that the ERP migration, cloud shifts, and upcoming data consolidation will shape future phases.

Our Understanding of Current Use Case

Based on discussions so far, here is our consolidated understanding:

Systems & Data

- Sage ERP is the primary system today, but planned to move to cloud in 6–7 months.
- Multiple systems spread across on-prem, older environments, and SQL Server.
- Some SAP dashboards exist but are not in scope today.
- The ecommerce platform is hosted in-house; cloud migration is future work.
- Product and customer data is fragmented across ERP, custom systems, and SQL databases.

AI Need & Immediate Scope

- Chatbot required **now** for: shipping policies, general information, FAQs
- No need for order-status integration in phase 1.
- No need for freight recommendations or order optimization in phase 1.
- Onboarding/verification workflow automation is a future module.

Our Understanding of Current Use Case Contd.

Phase-Wise Capability Expansion

Phase 1 – Foundation (Chatbot basics)

Future Phases :

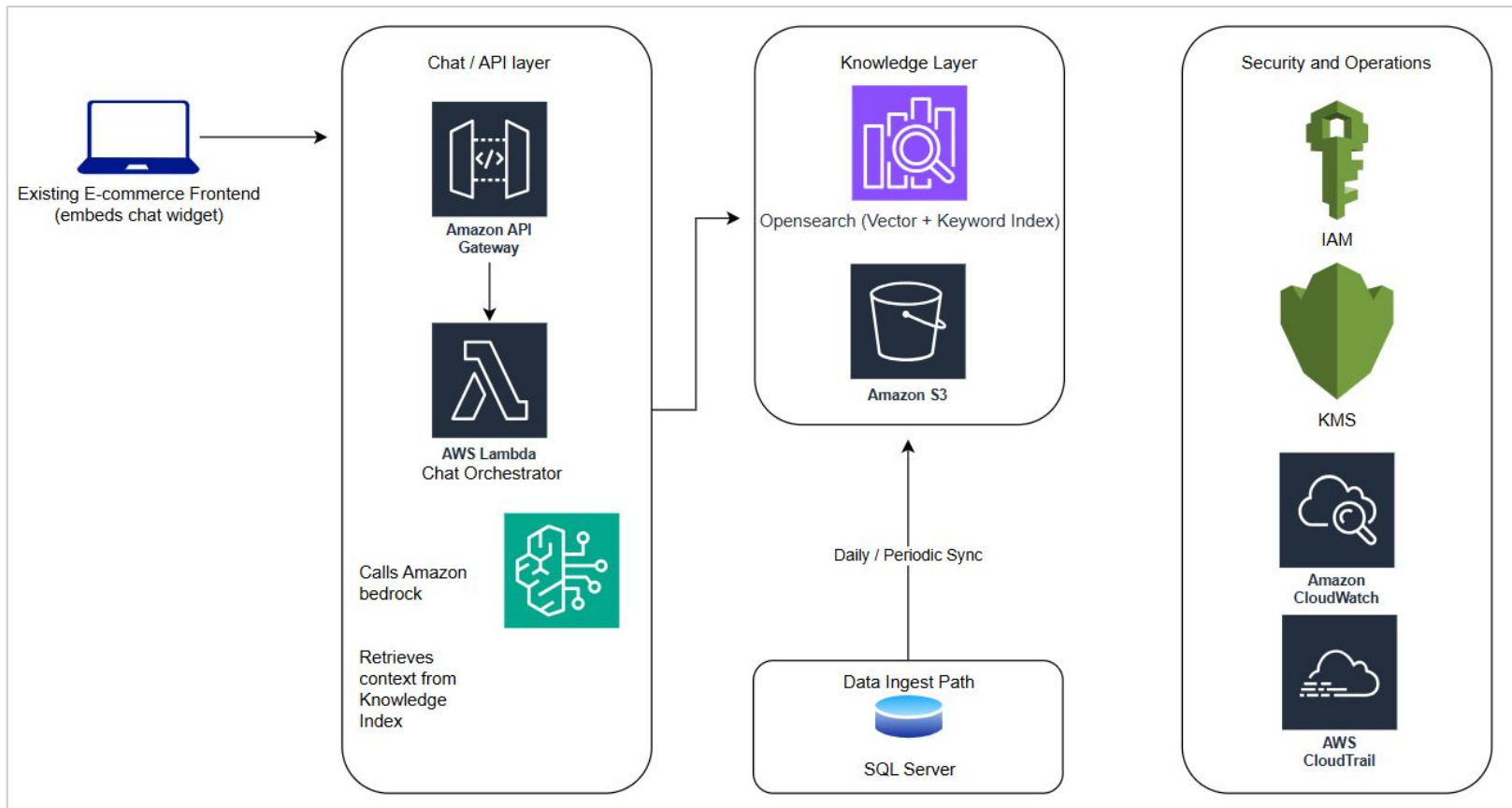
- Order-related queries
- Shopping assistant
- Freight & order optimization
- Onboarding, verification automation
- Data-driven custom bundles/packages

Baseline Environment Understanding

- Knowledge base for Bedrock will be manually curated by the customer and stored in SQL Server.
- Chatbot will be embedded inside existing home page.
- Escalation to human agent when bot cannot answer.
- Architecture should support future migration of ERP and web platform to cloud.

Technology Stack

Architecture Diagram



Application Layer

S.No.	Component	Technology
1	Chatbot Processing	Amazon Bedrock (Claude or similar) for natural-language understanding and response generation
2	Retrieval Layer	Amazon OpenSearch Serverless for vector search and contextual retrieval (RAG)
3	Knowledge Base Integration	SQL Server (customer-provided, curated knowledge base). Replicated/ingested into AWS using scheduled data sync or ETL job
4	Chatbot API Layer	AWS Lambda for request processing, query routing, and RAG orchestration
5	Frontend Integration	Embedded chatbot widget inside customer's existing web application (no new frontend hosted on AWS)

Infrastructure Layer

S.No.	Component	Technology
1	Compute	AWS Lambda for lightweight compute, low-latency responses
2	Networking	AWS VPC + VPC Endpoints for secure communication between Lambda and AWS Bedrock/OpenSearch
3	Storage	Amazon S3 for interim document dumps (if needed)
4	Database Connector	ETL pipeline or scripted sync to replicate SQL Server (on-prem) into AWS knowledge base periodically
5	Observability	Amazon CloudWatch for logs, metrics and error tracing specific to chatbot operations
6	IAM	AWS IAM policies specifically for Lambda, OpenSearch, and Bedrock access

Supporting Tools & Operations

S.No.	Component	Technology
1	Project Management	Zoho for project reporting
2	Communication	Google Meet for client and internal syncs
3	Bug/Issue Tracking	JIRA, Trello, Lean Testing, Mantis, Testlodge

Security and Compliance

- All AWS service interactions secured via IAM roles and scoped least-privilege access
- TLS encryption in transit for queries going from Lambda → Bedrock → OpenSearch
- KMS-managed encryption for OpenSearch/S3 data (phase one only applies if S3 used for snapshots)

Reliability

- Lambda auto-scales automatically to handle bursty query load
- OpenSearch Serverless scales storage and query capacity dynamically
- Built-in retry logic for intermittent API or query failures
- CloudWatch alarms set on Lambda errors, latency spikes, or OpenSearch query failures

Performance Efficiency

- RAG pipeline optimized to reduce hops: Lambda → OpenSearch → Bedrock
- Caching layer for high-frequency FAQ-type responses
- Minimal network overhead by hosting all processing inside AWS VPC

Operational Excellence

- Clear logging structure – request logs, retrieval logs, and error logs separated
- Monitoring dashboards (CloudWatch) for query volume, latency, and error rates
- Alerting configured only for critical items (e.g., failing sync jobs, OpenSearch errors)
- Ops runbook for handling chatbot failures or fallback to human intervention

Scope of Work

Chatbot Functional Capabilities (Phase 1 Only)

Core Chat Interface

- The chatbot must respond to user queries in natural language within the existing ecommerce web application's UI.
- The chatbot interface will appear as an embedded widget in the homepage and remain visible throughout the customer journey.
- It must support typed user queries (voice will NOT be included in this phase).
- Initial responses will be limited to pre-identified categories (To be confirmed during implementation):
 - Shipping policy
 - General business info
 - Store hours
 - Return policies
 - Membership requirements
 - Minimum order quantity guidelines
 - Payment methods
 - B2B qualification rules

Chatbot Functional Capabilities (Phase 1 Only)

Knowledge Base Integration

- Customer will consolidate all necessary documents, policies, FAQs, and static content into a unified SQL Server instance.
- Our team will mirror this data into a cloud-based searchable repository to avoid latency from direct on-prem calls.
- Data structure must follow consistent formats:
 - Policy documents
 - FAQ list
 - Product category definitions
 - Customer onboarding requirements (static info only; workflows not included yet)
- Chatbot must reference only the content provided for Phase 1; no external systems (ERP/OMS/warehouse/inventory) will be queried.

Chatbot Functional Capabilities (Phase 1 Only)

Response Logic

- Responses will be generated by the model using contextual references from the prepared knowledge base.
- If a user asks for information that requires order-level, customer-specific, or real-time system queries, the chatbot will gracefully redirect:
 - “I’m still learning to assist with that. Please reach out to customer support.”
- No personalization or order-specific intelligence will be included in this phase.

Language Handling

- Only English responses will be supported in Phase 1.
- Chatbot must understand common phrasing variants:
 - “Where is my order?”
 - “How long does shipping to Florida take?”
 - “Can I return items?”
 - “How do I qualify as a customer?”

Chatbot Functional Capabilities (Phase 1 Only)

Escalation to Human Agent

- If the chatbot cannot resolve a question after two clarifying prompts, it must:
 - Capture and log the user's intent summary.
 - Present a hand-off message: "I'm forwarding your request to a support specialist."
- Hand-off workflow:
 - Log the user query + timestamp into a predefined database table.
 - Notify the admin via email (process to be finalized with customer).
 - Provide the user with confirmation that their query has been forwarded.

Data Requirements & Handling

Data Sources in Scope

- Consolidated SQL Server knowledge base prepared by the customer:
 - Shipping rules
 - Policy documents
 - FAQs
 - Terms & conditions
 - Return and refund guidelines
 - Category-wise shipping variations (if provided)
- No real-time data from ERP/OMS/IMS/WMS will be accessed during Phase 1.

Data Requirements & Handling

Data Format Alignment

- Customer must provide all knowledge base content in consistent formats
- Our team will:
 - Validate completeness
 - Identify missing fields
 - Recommend data cleanup if detection of:
 - Inconsistent formats
 - Conflicting policy rules
 - Incorrect categorization

Data Refresh Process

- Daily or scheduled batch update process will be established between customer and our team.
- Customer will own the responsibility of updating the SQL Server knowledge base.
- Our solution will automatically update the cloud copy to keep the chatbot aligned.

Front-End Chatbot Widget Integration

Placement

- Chatbot will appear as a floating widget on the homepage of the ecommerce app.
- Widget expansion will display:
 - Chat area
 - Suggested starter prompts
 - Branding/customization based on customer theme colors

User Experience Requirements

- Widget must: Load without delay, Not block existing UI interactions
- Chat window must: Show time stamped messages, Allow scrolling history per session

Accessibility Requirements

- Readable font sizes
- Clear contrast between bot/user messages

Deliverables

- Fully functional AI-powered chatbot embedded into the client's existing e-commerce platform.
- Centralized knowledge base created from data collated in SQL Server (policies, FAQs, shipping rules, general info).
- Retrieval logic implemented so the chatbot answers based on structured knowledge content.
- Configuration for fallback-to-human workflow when the bot can't resolve a query.
- Initial test plan + functional validation with sample customer queries.

Assumptions

- Client will prepare and provide all required information in SQL Server (cleaned and consolidated).
- Client will expose required database access or provide data dumps in the agreed format.
- Chatbot will only handle general informational questions in Phase 1 (no OMS, freight, shopping assistant, or verification logic).
- Existing authentication, UI, and customer login flows remain unchanged and owned by the client.
- Integration with OMS/ERP or real-time systems will be part of later phases.

Out of Scope

- No workflow automation, onboarding verification, or third-party compliance checks.
- No integration with order management, freight systems, ERP, or shipping status APIs in this phase.
- No personalized recommendation engine or shopping assistant logic.
- No voice interface, multilingual support, or analytics dashboards.
- No e-commerce migration, UI redesign, or infrastructure modernization.

Team Structure & Timeline

Resources	Weeks			
	1	2	3	4
Project Manager				
Solution Architect				
Devops Engineer				
Frontend Dev				
Backend Dev				
UI/UX Designer				
Quality Analyst				
AI Engineer				

AWS Infra Cost Estimates (Monthly Recurring)

Growth Assumption :

Chatbot supporting following features - pulling real-time order information, offering product suggestions, optimizing freight decisions, generating offer bundles, assisting in customer onboarding, automating verification workflows

Environment	Estimated MRR (After 1 Year)
Production	\$1,418
Test	\$470
Dev	\$470
Total Estimated Monthly Cost	\$2,358

SMC (1 Year) : <https://calculator.aws/#/estimate?id=398d4c9012c8a40a59d186bbc33953990deb6b1e>

Professional Services Cost

Deliverables	Total Cost (USD)
- Fully functional AI-powered chatbot embedded into the client's existing e-commerce platform. - Centralized knowledge base created from data collated in SQL Server (policies, FAQs, shipping rules, general info). - Retrieval logic implemented so the chatbot answers based on structured knowledge content. - Configuration for fallback-to-human workflow when the bot can't resolve a query. - Initial test plan + functional validation with sample customer queries.	25,225
AWS POC Funding	4,244
Total Payable by the Client	20,981

**As an AWS Partner, we may be able to facilitate AWS funding to offset your project's costs or fees partially if you meet certain qualification criteria. Final approval is determined solely by AWS and is subject to various factors, such as the project's scope, industry, projected growth, and the intended use of AWS services.*

Payment Schedule

Phase	Amount
● Project kick off	50%
● Completion.	50%

Note: 1) Payment to be made within 15 days from the date of invoice

AWS Case Study

Customer	Aspect	Details
	About DoorDash	DoorDash is a leading last-mile delivery and local commerce platform serving tens of millions of consumers and millions of Dashers monthly.
	Challenges	High support volume across Dashers, merchants, and consumers; heavy reliance on manual support channels; latency issues; frequent escalations to human agents.
	Solution	Implemented a generative-AI self-service support system using Amazon Bedrock + Amazon Connect + Amazon Lex with RAG to interpret and respond to voice/chat queries in real time (<2.5 seconds).
	Outcome	Reduced agent transfers by ~49% in pilot cases; scaled to hundreds of thousands of automated calls per day; significantly reduced manual load and improved user satisfaction.

About Applify

Applify is a Global AWS Partner with strong expertise in GenAI-driven application modernization. We specialize in architecting intelligent, secure, and scalable solutions on AWS using Bedrock, and serverless technologies. Our approach focuses on transforming legacy products into future-ready, AI-integrated platforms that deliver real business outcomes while maintaining compliance and reliability.

We have helped many companies and organizations in building and launching successful products.





THANK YOU

For kickstarting your journey with us.